



POLITECNICO
MILANO 1863

Continual Learning Strategies and Avalanche

Federico Giannini

Ph.D. Student

DEIB – Politecnico di Milano

federico.giannini@polimi.it

Disclaimer

During this lecture, we will speak about Deep Learning and Neural Networks.

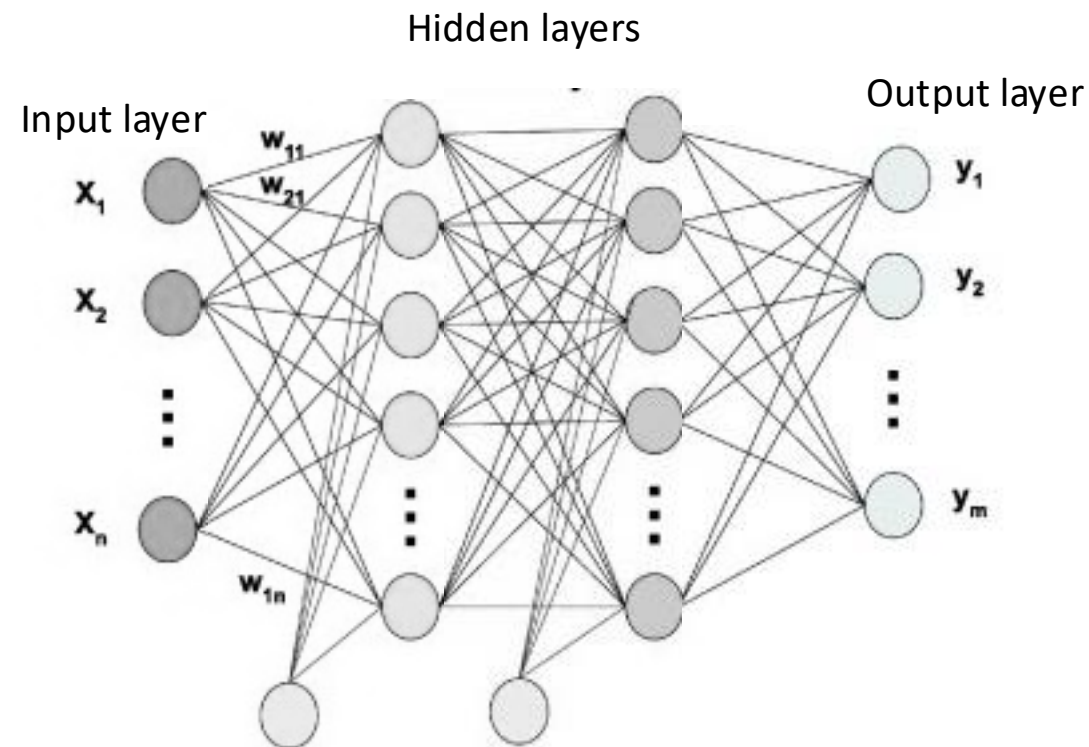
For the exam, you are not required to enter the details of each single strategy.

The goal is to understand the rationale and the general idea behind each strategy.



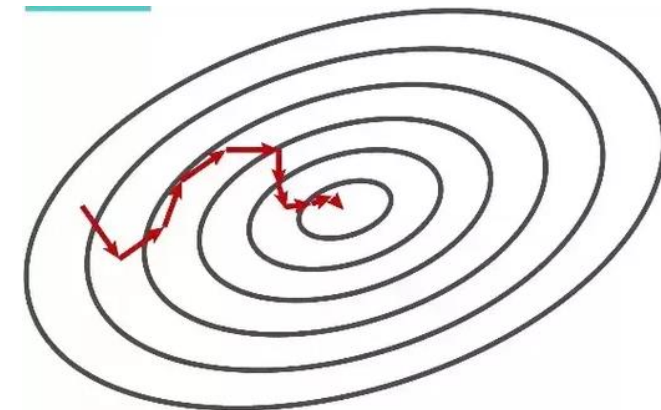
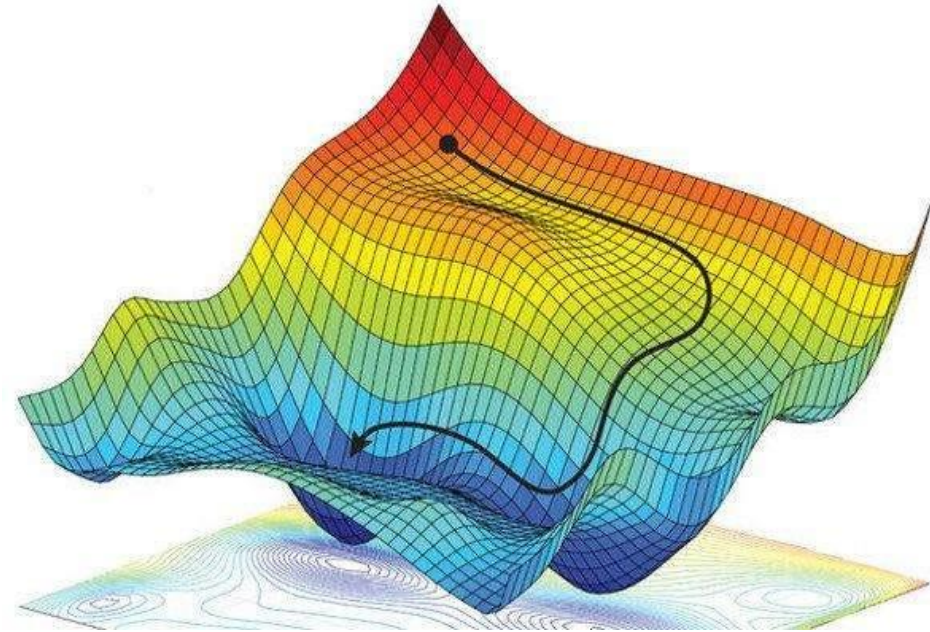
What do you need to know about Neural Networks?

- ▶ Composition of concatenated layers.
- ▶ Each layer is a set of neurons.
- ▶ Each hidden layer's neuron:
 - ▶ Receives the output of the previous layer's neurons as input.
 - ▶ Applies linear combinations of the inputs (weighted sum) and passes the results through a nonlinear activation function.
- ▶ The first part of the network implements feature engineering, producing a new feature space by mapping the original input through a nonlinear transformation. Each neuron represents a dimension in the new space.
- ▶ The output layer has one neuron for each class and returns the predicted probability for that class.



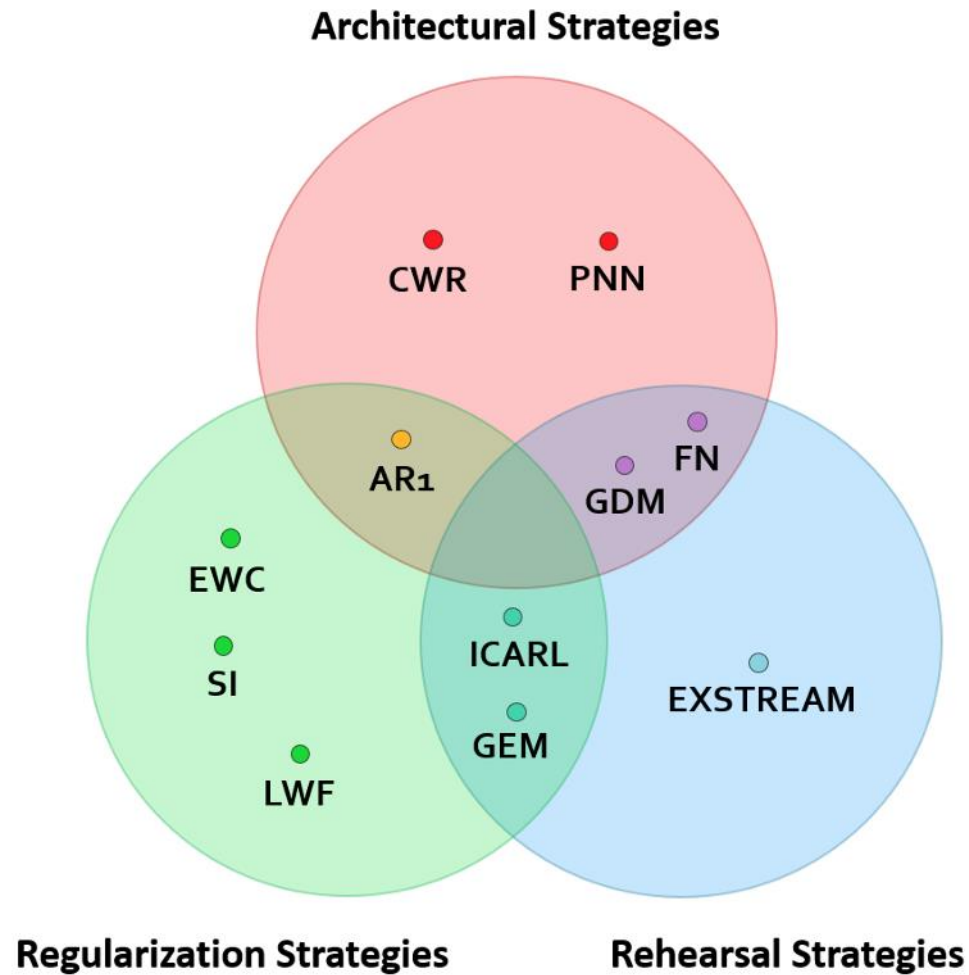
What do you need to know about Neural Networks training?

- ▶ The network's parameters (weights of the linear combinations) are adjusted to minimize a loss function, which measures the prediction error. It reaches its minimum when the model assigns probability 1 to the correct label and 0 to the others.
- ▶ Training set is divided into mini-batches.
- ▶ For each minibatch:
 - ▶ Evaluate the loss function.
 - ▶ Update weights along the gradient's direction (steepest descent).
- ▶ Training means finding the best weight configuration.
- ▶ At some point, we will find a local minimum of the loss function.



- ▶ **Replay:** during each experience, they mix old examples with new ones to not forget old tasks. Generative replay strategies generate examples associated with old tasks. The training set's size increases with the number of tasks.
- ▶ **Regularization:** they change the loss function by adding regularization terms to the new task's problem and not losing the old ones.
- ▶ **Architectural:** they introduce new parameters to the network (e.g. shared layers between all tasks and tasks-specific layers, transfer learning or weight masks). For some strategies, the number of parameters could increase with the number of tasks.

The most popular CL strategies



Baselines:

► Naïve strategy

It simply continues the training on the new experience.

► Cumulative

Each experience's training concatenates the current experience's training set with all the previous ones.

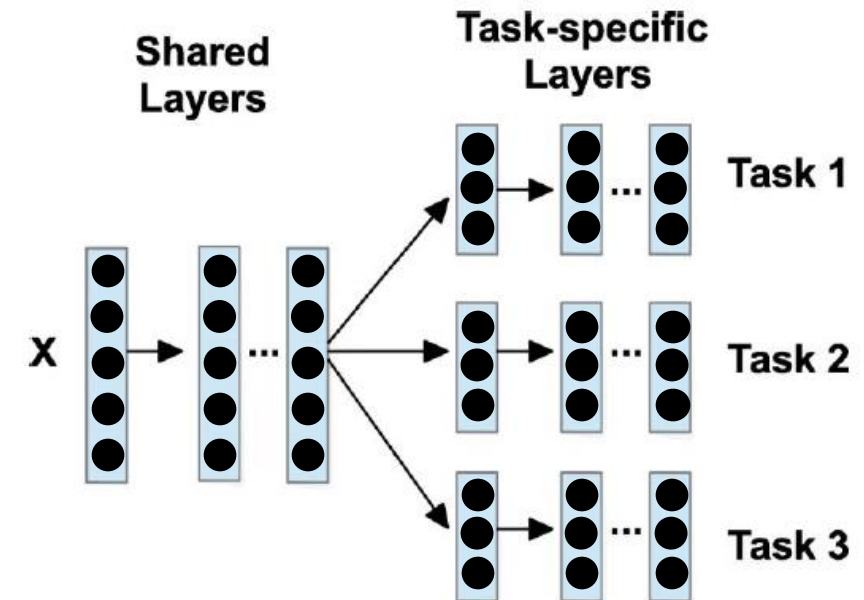
► Joint strategy (Offline)

It trains the model on all the experiences' training sets together.

Task Incremental Learning: Multi-Head models

In Task Incremental Learning, a possible design choice is to use a Multi-Head model.

- ▶ It adds a new head for each new task (task-specific layers with specific weights).
- ▶ There are parameters associated with old tasks.
- ▶ There are parameters associated with the new task.
- ▶ The net has also shared parameters.
- ▶ It requires the task label to select the correct head (some approaches try to recognize the tasks using an Autoencoder).



Replay: Random Replay

- ▶ It uses a fixed-size Random Memory (RM) to store a subset of random previous experiences' data points.
- ▶ During the training on the i -th experience, it trains the model on the i -th training set shuffled with RM.
- ▶ RM contains a random subset of the data points of the previous experiences' training sets.
- ▶ After the training, it randomly substitutes some data points with a random subset of the current experience's training set in RM.

Algorithm 1 Pseudocode explaining how the external memory RM is populated across the training batches. Note that the amount h of patterns to add progressively decreases to maintain a nearly balanced contribution from the different training batches, but no constraints are enforced to achieve a class-balancing.

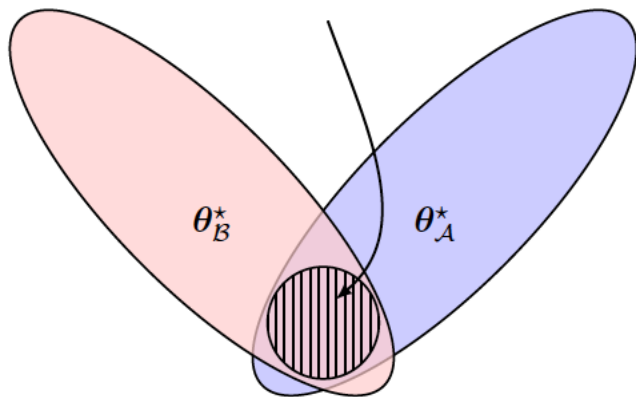
```
1:  $RM = \emptyset$ 
2:  $RM_{size}$  = number of patterns to be stored in  $RM$ 
3: for each training batch  $B_i$ :
4:   train the model on shuffled  $B_i \cup RM$ 
5:    $h = \frac{RM_{size}}{i}$ 
6:    $R_{add}$  = random sampling  $h$  patterns from  $B_i$ 
7:    $R_{replace} = \begin{cases} \emptyset & \text{if } i == 1 \\ \text{random sample } h \text{ patterns from } RM & \text{otherwise} \end{cases}$ 
8:    $RM = (RM - R_{replace}) \cup R_{add}$ 
```



Regularization: Elastic Weight Consolidation (EWC) – 1/2

- ▶ It adds a regularization term to the loss to penalize changes on important parameters for previous tasks.
- ▶ The idea is to search for the optimal configuration lying in the overlapping of different tasks' solution spaces.

Overlapping space that works for both tasks $\mathcal{A}$ and $\mathcal{B}$

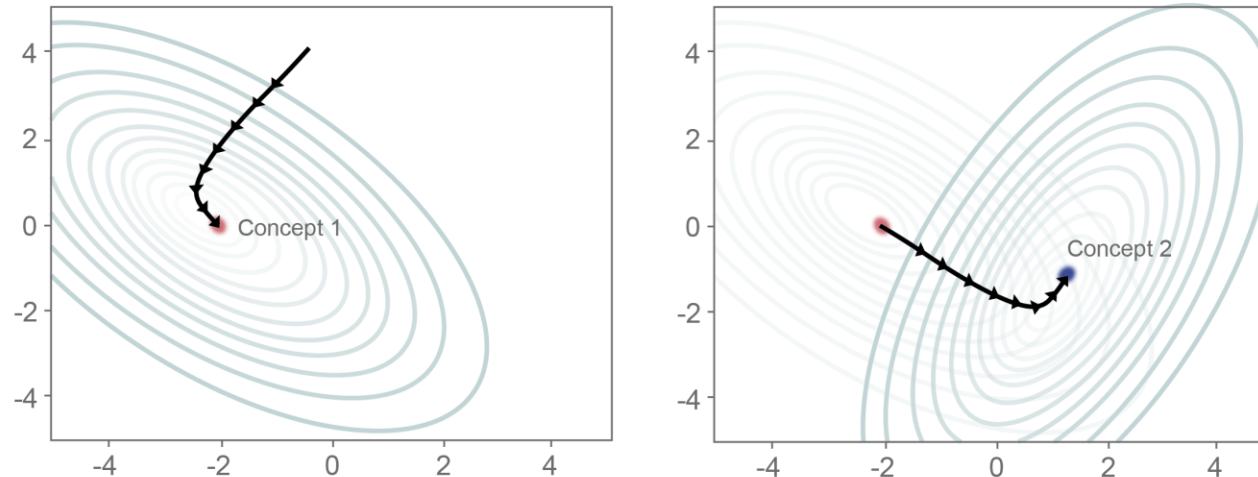


$$\mathcal{L}(\theta) = \overset{\text{Loss on task B}}{\mathcal{L}_B(\theta)} + \sum_i \frac{\lambda}{2} \overset{\substack{\text{Importance index of} \\ \text{parameter } i \text{ for task A}}}{F_i} (\theta_i \underset{\substack{\text{Distance between the current} \\ \text{value of parameter } i \text{ and the} \\ \text{optimal one for task A.}}}{-} \theta_{A,i}^*)^2$$

Adding the weighted distance means that we want to minimize it for important parameters.

Regularization: Elastic Weight Consolidation (EWC) – 2/2

- ▶ Optimizing very different tasks together could result in not finding a good trade-off:
 - ▶ Some tasks may perform well, while others badly.
 - ▶ In the worst scenario, we could find a solution that does not fit all the tasks.
- ▶ EWC results, in fact, in a tug-of-war over the direction of change of each task.



Regularization: Learning without Forgetting (LwF) – 1/2

- ▶ It is usually applied to Multi-Head models.
- ▶ When learning a new task, it uses Knowledge Distillation (KD) to reproduce the outputs of the previous tasks on the new one's data.
- ▶ The new training objective is:

$$\min_{\theta_s, \theta_o, \theta_n} (\lambda \mathcal{L}_{old}(Y_o, \hat{Y}_o) + (1 - \lambda) \mathcal{L}_{new}(Y_n, \hat{Y}_n) + \mathcal{R}(\theta_s, \theta_o, \theta_n))$$

KD Loss for the old task.

Given task o and the current task's data point: Y_o is the output returned by the head associated with o of the previous task's model. $\hat{Y}_o$ is the output returned by the head associated with o of the current model.

Weighted sum's coefficient that indicates the relevance of the KD loss.

Current task's Loss.

Y_n is the real target value of the current task's data point, $\hat{Y}_n$ is the predicted value by the last head.

Additional Regularization Term



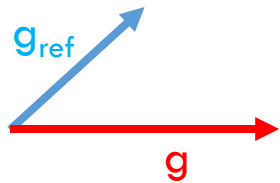
Regularization: Learning without Forgetting (LwF) – 2/2

- ▶ The loss function combines two objectives:
 - Reproducing the old tasks' outputs on the previous heads.
 - Learning the new task's classifications.
- ▶ It does not require storing the previous data (it only uses the current task's dataset).
- ▶ It only requires storing the previous model's predicted labels on the current dataset before starting the training.
- ▶ A specific version also deals with a single head and no task labels.



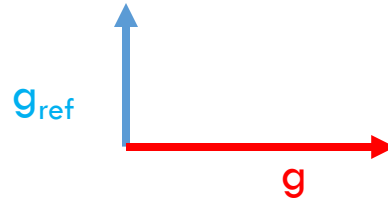
Replay+Regularization: Average Gradient Episodic Memory (AGEM)

- ▶ At every training step (mini-batch) it ensures that the average loss on the old tasks does not increase when learning the current one.
- ▶ It randomly chooses a mini-batch from the buffer and computes the gradient $\mathbf{g}_{\text{ref}}$ on it.
- ▶ It computes the gradient $\mathbf{g}$ on the current mini-batch.



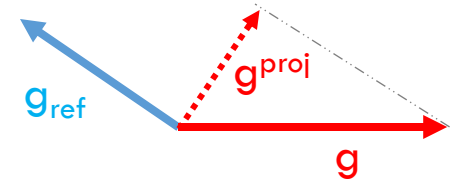
$$\mathbf{g}^T \mathbf{g}_{\text{ref}} > 0$$

$\mathbf{g}_{\text{ref}}$ and $\mathbf{g}$ have the same sense. Moving according to $\mathbf{g}$ in the weights' space also decreases the loss associate with the buffer.



$$\mathbf{g}^T \mathbf{g}_{\text{ref}} = 0$$

$\mathbf{g}_{\text{ref}}$ and $\mathbf{g}$ are orthogonal. Moving according to $\mathbf{g}$ in the weights' space does not alter the loss associate with the buffer.



$$\mathbf{g}^T \mathbf{g}_{\text{ref}} < 0$$

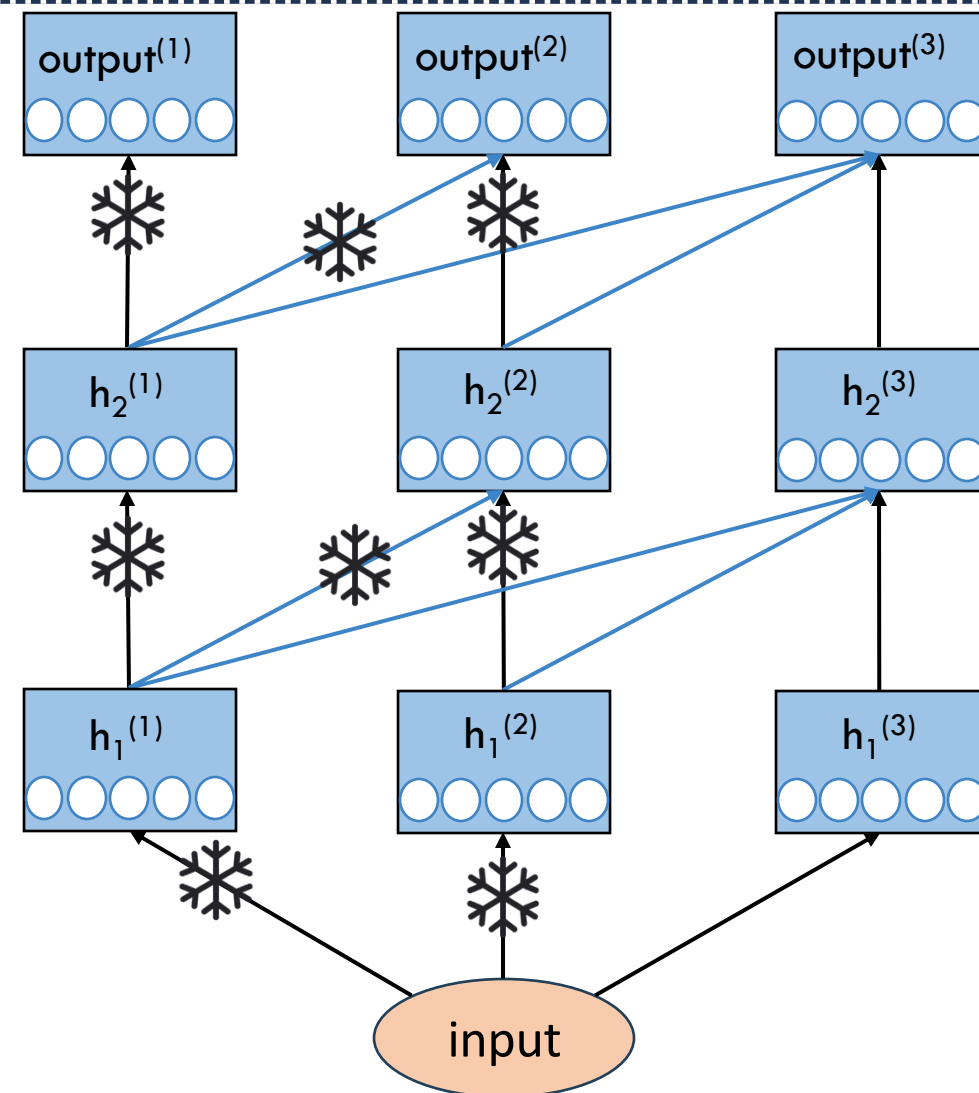
$\mathbf{g}_{\text{ref}}$ and $\mathbf{g}$ have different senses. Moving according to $\mathbf{g}$ in the weights' space increases the loss associate with the buffer.

If $\mathbf{g}^T \mathbf{g}_{\text{ref}} \geq 0$, it uses $\mathbf{g}$ for the gradient update. Otherwise, it projects $\mathbf{g}$ to have: $\mathbf{g}^T \mathbf{g}_{\text{ref}} = 0$ and produces $\mathbf{g}^{\text{proj}}$



Architectural: Progressive Neural Networks (PNN)

- ▶ It starts with a single Neural Network (column).
- ▶ Whenever a new task appears:
 - It freezes the parameters of the old columns.
 - It adds a new column.
- ▶ The hidden layer i ($i > 1$) of the column k receives:
 - The output of the hidden layer $i-1$ of k .
 - The outputs of the hidden layers $i-1$ of all the previous columns j ($j < k$).
- ▶ It uses transfer learning to combine the previous knowledge with the current one.
- ▶ It freezes old columns to avoid forgetting.
- ▶ As the Multi-Head models, it needs the task label.

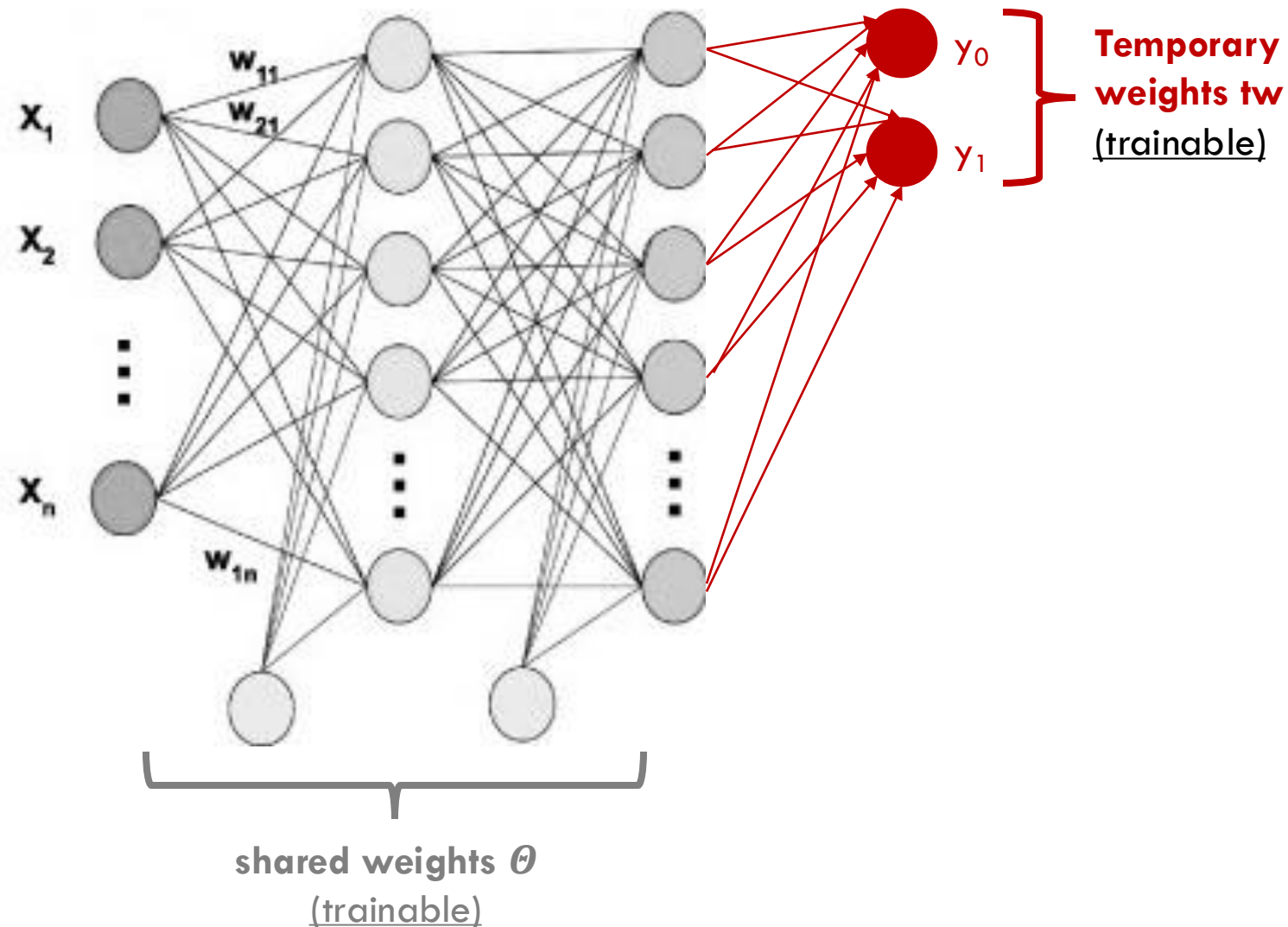


Architectural: Copy Weights with Re-init+ (CWR+)

x : 
 y : 0 1
 t : Null

Experience 1

time

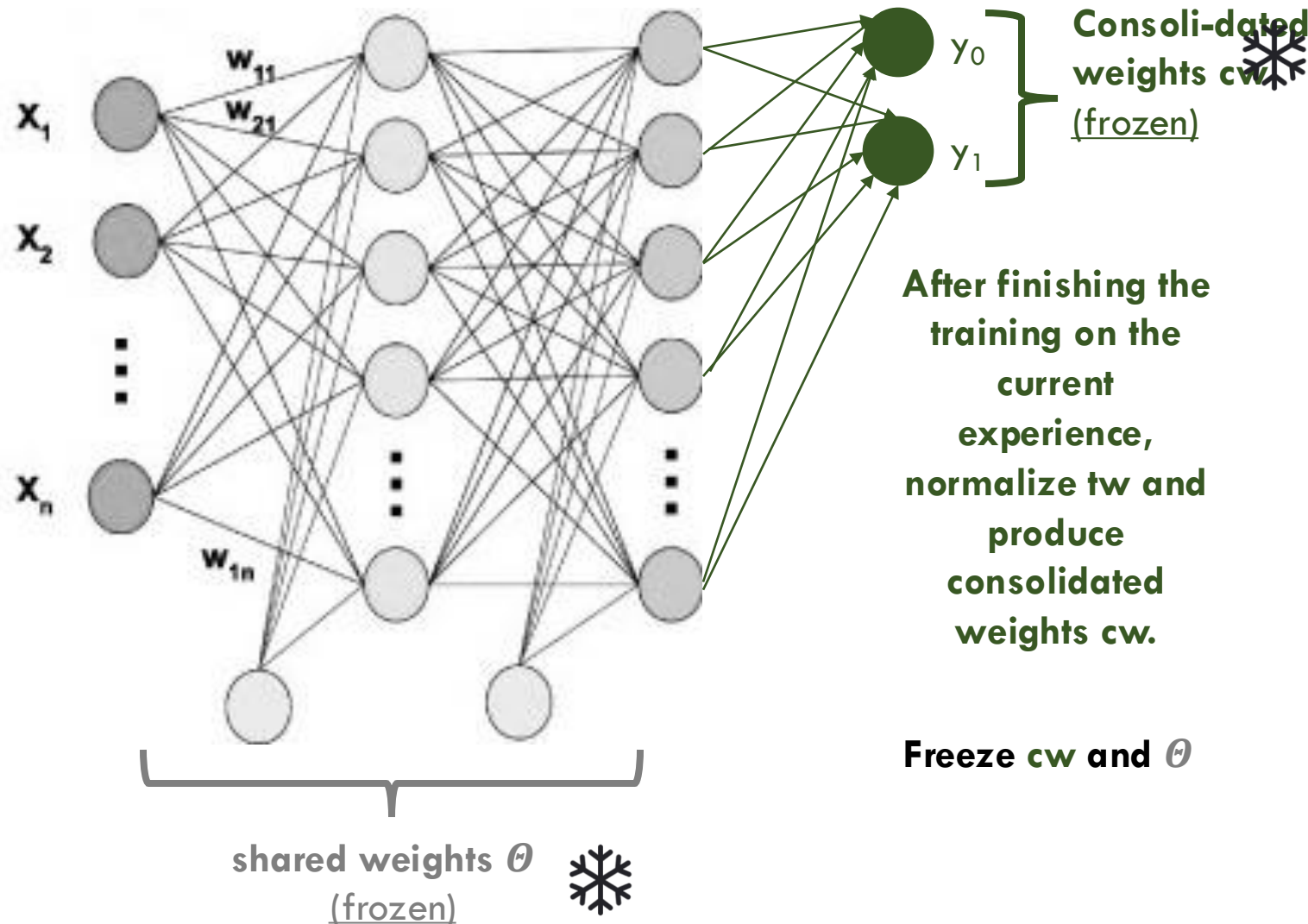


Architectural: Copy Weights with Re-init+ (CWR+)

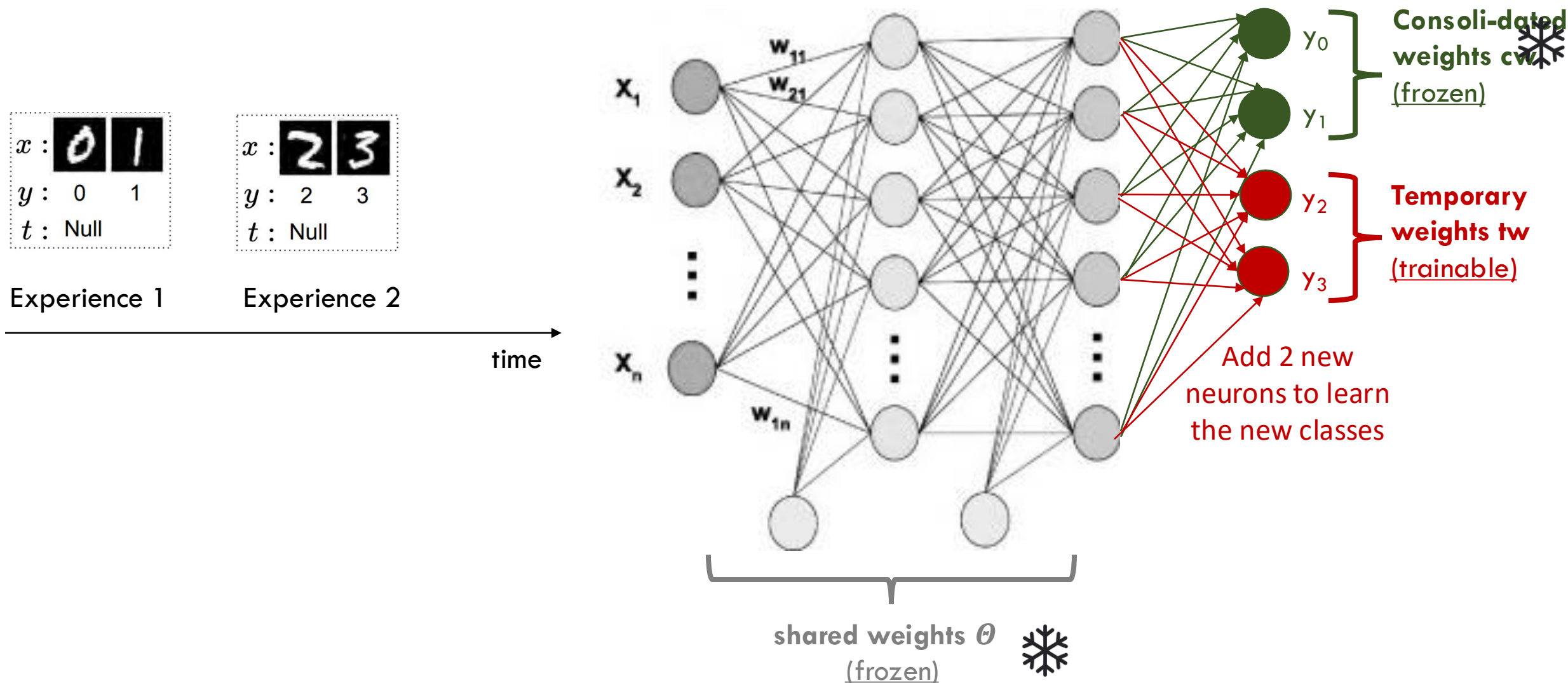
x :  
 y : 0 1
 t : Null

Experience 1

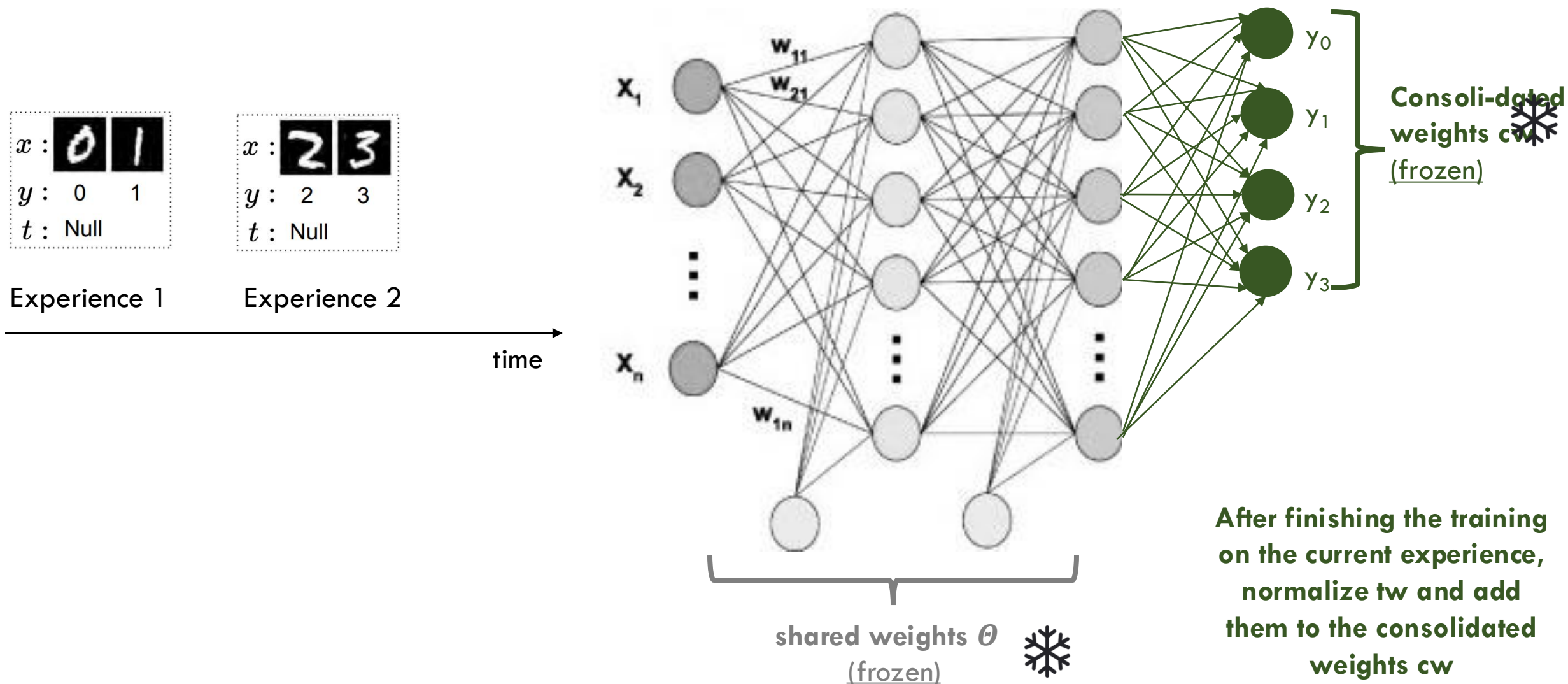
time →



Architectural: Copy Weights with Re-init+ (CWR+)



Architectural: Copy Weights with Re-init+ (CWR+)



Architectural: Copy Weights with Re-init+ (CWR+)

- ▶ It's meant to deal with **Class Incr. Scenario**.
- ▶ The network has **shared weights** Θ that are trained only on the first experience and then frozen.
- ▶ During training, it uses the output layer's **temporary weights** tw .
- ▶ During inference, it uses the output layer's **consolidated weights** cw .
- ▶ On each new experience's training:
 - For each new class, it adds a new neuron to the output layer. It re-init to 0 all the **tw**.
 - It trains **tw** on the new experience (while keeping Θ frozen from the second experience onward).
 - At the end of the training, for each class j in the experience, it saves the consolidated weights **cw** (the tw version normalized by subtracting the mean)

Algorithm 2 CWR+

```
1:  $cw = 0$ 
2: init  $\bar{\Theta}$  random or from a pre-trained model (e.g. ImageNet)
3: for each training batch  $B_i$ :
4:   expand output layer with  $s_i$  neurons for the new classes in  $B_i$ 
5:    $tw = 0$  (for all neurons in the output layer)
6:   Train the model with SGD on the  $s_i$  classes of  $B_i$ :
7:     if  $B_i = B_1$  learn both  $\bar{\Theta}$  and  $tw$ 
8:     else learn  $tw$  while keeping  $\bar{\Theta}$  fixed
9:   for each class  $j$  among the  $s_i$  classes in  $B_i$ :
10:     $cw[j] = tw[j] - \text{avg}(tw)$ 
11:   Test the model by using  $\bar{\Theta}$  and  $cw$ 
```

A crucial problem

Hyperparameter tuning and model selection are still open issues in CL.

- ▶ Use a first set of experiences.

The selected model is reset after selecting the best configurations, and a new training starts from the following experience.

- ▶ It does not learn the first experiences.
- ▶ Hyperparameters tuned and selected models on the first experiences may not fit the following.
- ▶ Model selection and hyperparameters tuning on the same stream and evaluation test sets (violation of Statistical Learning Theory principles).



Avalanche

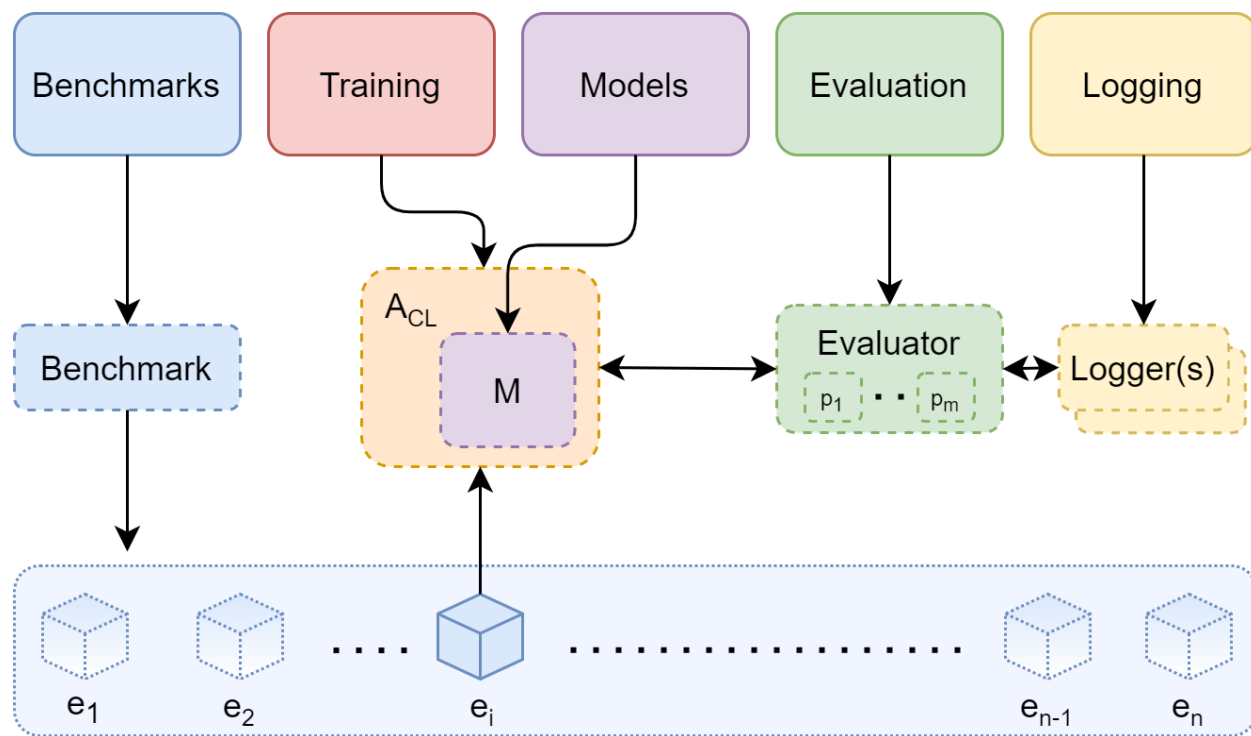
It is an end-to-end library based on Pytorch to provide a codebase for CL:

- ▶ fast prototyping
- ▶ training
- ▶ reproducible evaluation of algorithms.

Born within ContinualAI to provide a shared and collaborative open-source codebase.



Avalanche's Modules



► **Benchmarks**: it provides data stream generation. It contains all the standard datasets.

► **Training**: it provides model training. Implementing new CL strategies and using a set of pre-implemented CL baselines and state-of-the-art algorithms is possible.

► **Evaluation**: it provides all the utilities and metrics that can help evaluate.

► **Models**: it contains several model architectures and pre-trained models.

► **Logging**: it includes advanced logging and plotting features, including native stdout, file, and Tensorboard support.

And now... **Let's have some fun!**

